A 无穷无尽的力量

直接输出即可。

```
void solve() {
    int n;
    cin >> n;
    cout << string(n, 'a') << 'b' << string(n, 'a') << "\n";
}
```

B 无穷无尽的字符串

显然有 $\lfloor \frac{r-l}{3} \rfloor$ 个周期，剩下的字母我们可以通过检查 $r\%3$ 和 $(l-1)\%3$ 的关系确定是否检查到刚刚已经计算过的区间。

```
void solve() {
    int l, r;
    cin >> l >> r;
    l--;
    int a, b, c;
    a = b = c = (r - l) / 3;
    while (r % 3 != l % 3) {
        a += r % 3 == 1;
        b += r % 3 == 2;
        c += r % 3 == 0;
        r--;
    }
    cout << a << " " << b << " " << c << "\n";
}
```

C 无穷无尽的力量2.0

记 n 为短边， m 为长边，马的移动为 $(\pm1,\pm2)$ 或 $(\pm2,\pm1)$ 。

当 $n==1$ 时，显然跳不了，答案为 1 。

当 $n==2$ 时，显然只能上下跳，答案为 $(m-1)/2+1$。

当 $n==3$ 且 $m==3$ 时，除了中间的点其他都可以任意跳。

剩余情况，每一个点都是可达的。

注意 $10^9\times10^9=10^{18}$ 超 int 范围。

```
void solve() {
    ll n, m;
    cin >> n >> m;
    if (n > m) swap(n, m);
    if (n == 1) cout << 1 << "\n";
    else if (n == 2) cout << (m - 1) / 2 + 1 << "\n";
    else if (n == 3 && m == 3) cout << 8 << "\n";
    else cout << n*m << "\n";
}
```

D 无穷无尽的小数

$1 \le n, m \le 100$，并且 $1 \le k \le 10^4$，不觉得这是一件非常诡异的事情嘛。

显然肯定有长度为 $n \times m$ 的循环节，把两个数扩展到 $n * m$ 的长度，在 a 前加个 9， b 前加个 0，直接做高精度减法即可。

注意初始的 $borrow$ 为 $[a[0] < b[0]]$，需要考虑下一个循环有没有借 1。

```
void solve() {
    int n, m;
    cin >> n >> m;
    string a, b;
    cin >> a >> b;
    string A = "", B = "";
    for (int i = 0; i < 2 * m; ++i) A += a;
    for (int i = 0; i < 2 * n; ++i) B += b;
    A += '9';
    B += '0';
    int brw = a[0] < b[0];
    string ans = "";
    for (int i = n * m - 1; i >= 0; --i) {
        A[i] -= brw;
        brw = 0;
        if (A[i] < B[i]) brw = 1;
        int val = brw * 10 + A[i] - B[i];
        ans += (char)(val + '0');
    }
    reverse(all(ans));
    cout << n*m << "\n";
    for (int i = 0; i < n * m; ++i) cout << ans[i];
    cout << "\n";
}
```

E 无穷无尽的树

叶子节点:

- 如果 u 是原树的叶子，它对答案没有贡献。
- 返回一个无效状态，例如 $(-1, 0)$。
- 当前节点的答案输出 0。

非叶子节点:

- u 本身不是叶子，所以 u 会留在树中。初始状态设为 $(dep[u], 1)$。
- 遍历所有子节点 v，获取 v 返回的 (d, c)。

状态转移:

- 如果子树返回的深度 $d >$ 当前记录的 max_dep： 更新 $max_dep = d$，$cnt = c$。
- 如果子树返回的深度 $d ==$ 当前记录的 max_dep： 累加 $cnt+ = c$。
- 如果子树返回的深度 $d < max_dep$： 忽略。
- 记录当前节点的答案为 cnt。

- 将合并后的 (max_dep, cnt) 返回给父节点。

```cpp
void solve() {
    int n;
    cin >> n;

    vector<vector<int>> adj(n + 1);
    vector<int> dep(n + 1), ans(n + 1);
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].pb(v);
        adj[v].pb(u);
    }

    function<PII(int, int, int)> dfs = [&](int u, int p, int d) {
        dep[u] = d;
        bool f = true;

        int mx = d;
        int cnt = 1;

        for (int v : adj[u]) {
            if (v == p) continue;
            f = false;

            PII res = dfs(v, u, d + 1);

            if (res.fi == -1) continue;

            if (res.fi > mx) {
                mx = res.fi;
                cnt = res.se;
            } else if (res.fi == mx) {
                cnt += res.se;
            }
        }

        if (f) {
            ans[u] = 0;
            return make_pair(-1, 0);
        } else {
            ans[u] = cnt;
            return make_pair(mx, cnt);
        }
    };

    if (n == 1) {
        cout << "0" << "\n";
        return;
    }

    dfs(1, 0, 1);

    for (int i = 1; i <= n; ++i) {
```

```
        cout << ans[i] << " ";
    }
    cout << "\n";
}
```

F 无穷无尽的数

这个区间 $[l, r]$ 在结构上由三部分组成：

- 头部：x 的某个后缀（从下标 $(l-1)\%n$ 开始到 $n-1$）。
- 中间：k 个完整的 x。
- 尾部：x 的某个前缀（从 0 开始到 $(r-1)\%n$）。

我们可以预处理前缀，令 $H[i]$ 表示 $x[0 \ldots i-1]$ 的数值，则子串 $x[i \ldots j]$ 的数值可以通过 $(H[j+1] - H[i] \times 10^{j-i+1}) \pmod{M}$ 得到。

对于中间部分，实际上是一个等比数列，公比是 10^n。

最终答案为 $\text{头部} \times 10^{\text{中间长度}+\text{尾部长度}} + \text{中间} \times 10^{\text{尾部长度}} + \text{尾部}$

```
void solve() {
    ll n, l, r;
    cin >> n >> l >> r;
    string s;
    cin >> s;
    vector<Z> p(n + 1), h(n + 1);

    p[0] = 1;
    for (int i = 0; i < n; ++i) {
        p[i + 1] = p[i] * 10;
        h[i + 1] = h[i] * 10 + (s[i] - '0');
    }

    auto g = [&](int i, int j) {
        return h[j + 1] - h[i] * p[j - i + 1];
    };

    ll u = (l - 1) / n, v = (r - 1) / n;
    int x = (l - 1) % n, y = (r - 1) % n;

    if (u == v) {
        cout << g(x, y) << "\n";
        return;
    }

    ll k = v - u - 1;
    Z mid = 0;

    if (k > 0) {
        mid = h[n] * (ksm(Z(10), n * k) - 1) / (ksm(Z(10), n) - 1);
    }

    cout << g(x, n - 1)*ksm(Z(10), n * k + y + 1) + mid*ksm(Z(10), y + 1) +
g(0, y) << "\n";
```

```
}
```

头文件

```
//Another
#include<bits/stdc++.h>
#include<bits/extc++.h>
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
using namespace std;

typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
typedef __int128 i128;
typedef pair<int, int> PII;
typedef pair<ll, ll> PLL;
typedef tuple<ll, ll, ll> TLLL;
typedef __gnu_pbds::tree<PLL, __gnu_pbds::null_type, less<PLL>, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> Tree;
// typedef __gnu_pbds::tree<ll, __gnu_pbds::null_type, less<ll>, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> Tree;

constexpr int inf = (ll)1e9 + 7;
constexpr ll INF = (ll)2e18 + 9;
// constexpr ll INF = (ll)4e18;
// constexpr ll MOD = 1e9 + 7;
constexpr ll MOD = 998244353;
constexpr ld PI = acos(-1.0);
constexpr ld eps = 1e-10;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ull randint(ull l, ull r) {uniform_int_distribution<unsigned long long> dist(l, r); return dist(rng);}

void init() {

}

void solve() {

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    init();

    int t = 1;
```

```
    // cin >> t;
    for (int _ = 1; _ <= t; ++_) {
        solve();
    }
    return 0;
}
```

自动取模

```
template<class T>
constexpr T ksm(T a, ll b) {
    T res = 1;
    while (b) {
        if (b & 1) res *= a;
        b >>= 1;
        a *= a;
    }
    return res;
}

template<int P>
struct MInt {
    int x;
    constexpr MInt(): x() {}
    constexpr MInt(ll x) : x{norm(x % getMod())} {}
    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) {
        Mod = Mod_;
    }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const {
        return x;
    }
    explicit constexpr operator int() const {
        return x;
    }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
```

```
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return ksm(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & {
        return *this *= rhs.inv();
    }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr istream &operator>>(istream &is, MInt &a) {
        ll v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr ostream &operator<<(ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
```

```
};

template<>
int MInt<0>::Mod = 998244353;

using Z = MInt<MOD>;
```